

Mapeamento Visual de Arquiteturas de softwares C++ Orientado a Objetos

Hugo Rodrigues Torquato^{1*}; Heinrich da Solidade Santos²

¹ Engenheiro de Software II. Mestre. Rua lavras 480 – São Pedro; 3033010 Belo Horizonte, Minas Gerais, Brasil.

² USP/ESALQ. Doutor. Av. Pádua Dias, 11 - Agronomia; 13418-900 Piracicaba, São Paulo, Brasil

*autor correspondente: hugortorquato@gmail.com

Título do trabalho de conclusão de curso

Introdução

No contexto de programação orientada a objetos, as abstrações são fundamentais para organização e atribuição de responsabilidades. A compreensão da lógica é simples quando os assuntos são comumente conhecidos e/ou com poucos níveis de abstrações. Entretanto, o código se torna mais complexo e, por consequência, demanda mais esforço e tempo para compreendê-lo quando se trata de uma base de código extensa, desenvolvida durante vários anos, por vários profissionais e com foco em um conhecimento específico. Em uma ótica empresarial, este cenário implica em maiores investimentos para que novos engenheiros sejam incorporados à equipe por causa da complexidade, e adiciona um nível de abstração extra nas tomadas de decisão relacionadas ao desenvolvimento do *software* por não ter uma visão geral do todo. Sob essa perspectiva, visualizar a organização de uma base de código clara e intuitiva agrega valor, principalmente, na rotina das empresas desenvolvedoras de *softwares*.

A visualização e organização de informações de um *software* auxilia no seu gerenciamento, o que, de forma secundária, reflete na maior manutenibilidade do produto. Martin (2009) ilustra essa dinâmica ao comparar produtividade pelo tempo de existência do *software*: ao passo que a extensão e complexidade do código aumentam, especialmente sem planejamento, as chances de se transformar em um emaranhado de soluções improvisadas também aumentam. Nesse cenário, o autor argumenta que a produtividade para desenvolver novas funcionalidades e a manutenção do código declina drasticamente a médio e longo prazo, quando não gerenciado corretamente, tornando-se quase impraticável e economicamente inviável.

O ponto de partida para projetos dessa natureza contempla a análise de código fonte, tema de investigação consolidado para diversas áreas na computação: a otimização de compilador, abordado por Dean (1995), que expõe os benefícios ao realizar análises de hierarquia de classes como método para resolver ambiguidades em invocações de funções virtuais; análises de qualidade de *software*, abordado por Ceconello (2016), apresenta alguns dos problemas recorrentes na programação orientada a objetos; e estudos até mesmo no ramo da avaliação do quão eficiente estão os modelos de inteligência artificial baseados em linguagens (LLMs) quando comparado programação funcional com orientada a objetos, abordado por Wang (2024), que evidencia a necessidade de melhorias.

Inúmeras aplicabilidades, soluções e produtos foram propostos com objetivo de aprimorar a experiência durante e após o desenvolvimento de *software*. Silva (2023) detalha os benefícios gerais das ferramentas que podem ser integradas em diversos momentos no

ciclo de vida do *software*, desde a etapa de codificação até a validação do produto final. Como exemplos temos: o *SonarQube*, uma plataforma de análise de código com foco em qualidade e cobertura de testes; o *Clang Static Analyzer*, uma ferramenta de análise nativa do LLVM aplicada ao C e C++; e *AddressSanitizer*, como ferramenta para identificar erros de memória em tempo de execução.

Frente ao exposto, ferramentas de visualização e organização de uma base de código são bastante requisitadas em projetos de *software*. Por consequência, é tema de pesquisas tanto no âmbito acadêmico quanto no desenvolvimento de novos produtos e soluções no setor privado. A literatura relacionada a este projeto de pesquisa está associada com a investigação feita em nível de compiladores e nos ambientes de desenvolvimento integrado (IDEs), com o intuito de auxiliar o programador na escrita e otimização da compilação de código. Entretanto, o mapeamento estruturado para facilitar discussões e a tomada de decisão, não tem sido amplamente abordados e/ou mencionados em relevantes estudos e produtos. Sendo assim, se faz relevante arquitetar um software capaz de realizar uma análise estática em códigos baseados em programação orientada a objetos, extrair informação sobre a organização das classes e expor, de maneira simples e direta, informações estruturadas cerca do código em questão.

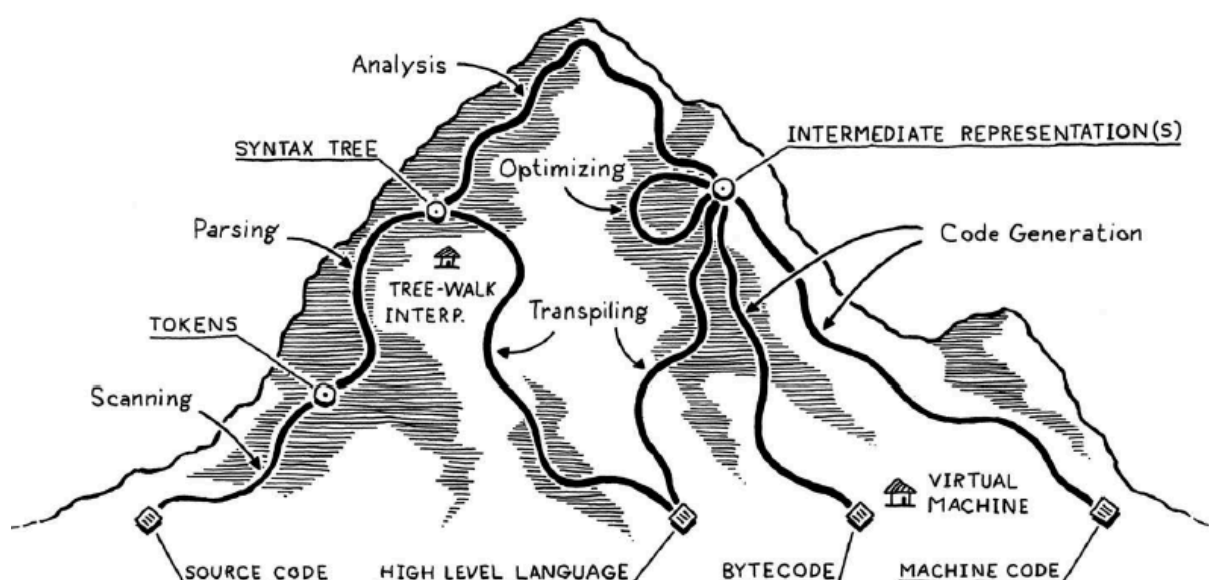
Metodologia ou Material e Métodos

O presente trabalho fundamenta-se na análise de código que, segundo a literatura, pode ser conduzida por uma abordagem tanto estática quanto dinâmica. Um comparativo é apresentado por Silva (2023), que ressalta tanto os aspectos positivos quanto os reveses de cada estratégia. O autor enfatiza o potencial positivo alcançado quando utilizadas em conjunto. No contexto deste projeto de pesquisa, a análise estática revela-se uma opção promissora por depender unicamente do código, sem a necessidade de compilá-lo. Essa característica torna o projeto proposto agnóstico em relação à linguagem de programação e permite avaliar a intenção original do desenvolvedor ao examinar o código antes de eventuais otimizações.

As etapas iniciais do projeto concentraram-se na implementação de elementos típicos aos de um compilador, com início no mapeamento do código fonte e evoluíram para a organização das informações em forma de *tokens*. Essa categoria de dados constitui a entrada para o processo de *parsing*, responsável por agrupar os *tokens* em expressões sintáticas mais abrangentes e viabilizar a construção das chamadas árvores de sintaxe. Essas, por sua vez, possibilitam a análise do código de acordo com as especificidades da

linguagem de programação utilizada. As informações necessárias para o projeto de pesquisa proposto foram coletadas nesta etapa, com ênfase na estrutura da árvore de sintaxe.

Embora existam outras etapas em um projeto de compilador, os resultados por elas gerados não agregam informações relevantes a este projeto. Ademais, Nystrom (2020) apresenta uma visão detalhada de todo o processo de desenvolvimento de um compilador, explorando as etapas: otimização do código fonte; tradução para linguagem de baixo nível mais próxima de máquina e compilação conforme a escolha do ambiente de execução do *software*. A Figura 1 apresenta, de forma simplificada, o fluxo completo das etapas de desenvolvimento de um compilador, sendo cada uma detalhada por Nystrom (2020). As etapas que interagem com o escopo deste trabalho são: *scanner*, *tokens*, *parsing* e *analysis*.



Possíveis caminhos de implementação de compiladores.

Figura 1. Ilustração dos possíveis caminhos de implementação de compiladores.

Fonte: Nystrom (2020)

Os autores Nystrom (2020), Aho (2007) e Cooper (2012) foram as referências principais para fundamentar o tema compiladores. Com foco nas etapas relevantes para o este trabalho, temos o mapeamento dos caracteres de um código como a primeira das três etapas fundamentais. Nesta fase inicial, a performance é crucial, uma vez que requerer uma análise sequencial de todos os caracteres dos arquivos fornecidos ao sistema. Todas as informações precisam ser categorizadas de forma eficiente para possibilitar o agrupamento dos caracteres em *tokens*, que constituem uma abstração de informações úteis para o interpretador. Em termos simplificados, o menor agrupamento de caracteres com significado, incluindo o tratamento de ambiguidades lexicais.

Além de percorrer todos os caracteres com o objetivo de identificar os *tokens*, a etapa do *scanner* também é responsável por associar essas informações à linha correspondente no arquivo fonte. O processo de identificação dos tokens então segue para a identificação dos *lexemes*, grupo de caracteres ainda sem significado. Após identificado, inicia-se o processo de compreensão do que o *lexeme* representa, utilizando expressões regulares para verificar se corresponde a uma palavra reservada, operador ou declaração de variável. Uma vez catalogado, um objeto *token* é criado para encapsular o conteúdo e demais informações relacionadas a essa porção de código.

A segunda fase é o *parser*, que determina se a sequência de *tokens*, gerada pelo mapeamento do código, constitui uma sentença válida e o insere em uma representação que emprega árvores de sintaxe para organizar as expressões lógicas. Obtém-se, como resultado intermediário, a tradução do código para uma linguagem livre de contextos que simplifica a interpretação do token utilizando de modelos de algoritmos já validados como: *top-down* e *bottom-up*. O *top-down* correlaciona as entradas dos *tokens* com as produções gramaticais, visando a eficiência ao prever a próxima palavra. O *bottom-up* opera com os detalhes, a precisa sequência de palavras é mantida em memória até que identificado o respectivo contexto.

Por fim, terceira fase contempla a análise de contexto, que, no âmbito dos compiladores, é responsável por realizar um processamento adicional ao *parser*, identificando erros funcionais com base na árvore de sintaxe. No contexto do projeto de pesquisa proposto, essa etapa é explorada de maneira limitada, restringindo-se à identificação de relações entre classes dentro do código analisado.

Diante do contexto apresentado e do panorama funcional do *software*, é pertinente discutir como ele está implementado. Em uma visão de alto nível, a fonte de dados é uma base de código e o resultado uma estrutura de dados no formato JSON para utilização posterior. Em uma análise detalhada, observa-se que é possível suportar várias linguagens de programação, mapear diversos tipos de *tokens* para cada uma das linguagens e performar diversos tipos de operações dependendo do *token* e da linguagem selecionada. O escopo deste projeto de pesquisa, no entanto, limita-se à análise de códigos escritos em C++, extraídos de repositórios no *GitHub*, considerando somente estruturas de classes. Entretanto, a escalabilidade para suportar outros tipos de linguagens e análises é considerada na arquitetura do *software*, que contempla as possíveis expansões para demais linguagens sem a necessidade de refatorações significativas na lógica principal, somente adição de novas implementações.

Na discussão da implementação da pesquisa, o padrão de projeto *visitor* é um exemplo que aborda o antagonismo entre inserção de tipos e operações em uma lógica.

Iglberger (2022) discorre sobre o tema com análises sobre a escolha do paradigma de programação frente à necessidade de adicionar tipos ou operações. Ao passo que em uma programação procedural seja simples adicionar operações novas, adicionar tipos tem por consequência alterar componentes base do código e propagar a mudança para as demais camadas. Em contrapartida, a programação orientada a objetos simplifica a adição de novos tipos, mas complica a inserção de novas operações. O padrão visitor atua, junto a orientação a objetos, para simplificar a inserção de operações ao delegar a implementação da operação para uma classe externa que será, em termos simples, aceita pela classe principal.

O *visitor* convencional transforma o estilo de programação orientado a objetos de simples inserções de tipos para difícil e de difícil inserção de operação para simples, mas existe uma alternativa que simplifica ambas inclusões, o *std::variant* do C++. Tal alternativa reserva em memória o espaço necessário para o maior tipo que o *visitor* pode operar e utiliza desse bloco de memória para definir a ação e a classe que será executada. A viabilidade da utilização do *std::variant* foi avaliada e, diante de seu potencial, está prevista adoção no projeto. O padrão *visitor* também é abordado por Nystrom (2020) no contexto do *parser*, ressaltando a simplicidade de adicionar novas operações em um conjunto de tipos específicos sem alterar a implementação da classe em questão.

Ainda na discussão da implementação, é fundamenta contar com uma estrutura de *logs* atrelada à toda etapa de desenvolvimento, sendo ela, inclusive, o ponto de partida da implementação. O padrão de projeto *Singleton* é amplamente questionado quanto à testabilidade, presença de estado global compartilhado e um acoplamento implícito entre diversas porções do *software*. Em contra partida, oferece uma grande simplicidade no controle de acesso, como em arquivos, garantindo que apenas uma instância esteja ativa. Iglberger (2022) questiona sobre os motivos desse padrão, originalmente classificado como padrão de projeto, ser caracterizado como padrão de implementação. Por ser uma maneira muito eficaz de limitar o número de instâncias de um objeto, focado em implementação, do que desacoplar ou gerenciar dependências, facado em projeto.

Outro padrão amplamente utilizado na implementação é o *strategy*, também discutido por Iglberger (2022). Este padrão define uma família de algoritmos semelhantes, encapsulando cada um e tornando-os intercambiáveis conforme o tipo do cliente. Uma analogia simples para compreender esse padrão é solicitar um software que desenhe uma forma geométrica, seja ela círculo, quadrado e triângulo. A estrutura proposta implementa uma função virtual pura com a função desenhar para todos os tipos, mas existem classes bases que herdam e implementam as especializações, cada qual para seu respectivo tipo. Assim, quando utilizado da classe virtual, o *software* resolve, em tempo de execução, qual

operação desenhar frente ao cliente que fez a requisição. Promovendo, assim, maior flexibilidade e desacoplamento na implementação.

No projeto, o padrão *strategy* foi útil para o tratamento de URLs com a biblioteca *curl*. Embora essa biblioteca tenha sido utilizada na implementação atual, ela pode ser facilmente substituída por outra, desde que respeite os contratos estabelecidos entre as partes e que receba os mesmos parâmetros de entrada e retornem os mesmos resultados. Possibilitando assim, uma alteração sem necessidade de refatoração de todo o fluxo que utiliza URLs. Este padrão também foi essencial na implementação dos testes e na injeção de dependências, pois permite a criação de objetos falsos (*mocks*) para simular comportamentos externos, facilitando o controle e a previsibilidade dos testes. Dessa forma, promove mais controle e simplicidade para criação dos testes unitários.

Outros padrões de projeto podem e devem ser utilizados, embora a identificação de todos os contextos e detalhes de implementação que justifiquem sua escolha ainda estejam em análise. À medida que validado os benefícios do padrão para o projeto e com base nas explicações fornecidas por Iglberger (2022), serão implementados.

Dentre a ampla gama de ferramental que poderia ser utilizado para a implementação deste projeto, a linguagem C++ foi selecionada pela performance e eficiência. A estrutura do *software* em formato de uma API RESTful segue a tendência em proporcionar praticidade na utilização, sem a necessidade de transferir e instalar localmente, simplesmente interagindo com o serviço e obtendo a informação desejada. Resultado disponibilizado em formato JSON é amplamente adotado em APIs e de simples conversão para diversas aplicações. A biblioteca em C++ utilizada para implementar a API é a *Crow*, e a escolha baseia-se no suporte ativo da ferramenta no *GitHub* e na presença de uma documentação elaborada.

Frente ao projeto proposto, foi fundamental associar a implementação de testes ao desenvolvimento do projeto para assegurar a qualidade e confiabilidade do *software*. Atrelado ao projeto de pesquisa foi pertinente desenvolver dois tipos de testes: unitários e de integração. Os testes unitários foram utilizados na validação de componentes da implementação, de forma isolada, explorando as particularidades e detalhes das lógicas implementadas. O *framework* escolhido para os testes unitários é o *Google Tests*, amplamente adotado na indústria e com fácil acesso à documentação. A metodologia de implementação dos testes segue o padrão do *Test-Driven Development (TDD)* abordado por Langr (2013), que ilustra os conceitos utilizando o *framework* com C++.

Embora fundamentais no desenvolvimento, os testes unitários não são suficientes para abranger a completude do algoritmo. Por isso, foram complementados por testes de integração para, em uma visão de alto nível, avaliar se o *software* se comporta da maneira projetada. Este modelo de teste consiste em prover entradas já conhecidas ao *software* e

avaliar o retorno obtido com o esperado para aquela determinada entrada. Essa combinação estratégica de testes explorou a granularidade dos testes unitários com a abrangência dos testes funcionais para garantir um *software* de melhor qualidade.

Dada a complexidade e amplitude dos temas envolvidos, a criação de um repositório versionado para orquestrar todo o desenvolvimento do projeto mostrou-se essencial. Vislumbra-se também a criação de uma rotina de integração constante (CI), para compilar e executar testes sempre que alguma alteração for inserida no repositório. Em um horizonte futuro, também vislumbra a implementação do despacho contínuo (CD) de atualização sempre que houver uma versão estável. Contudo, este foi um assunto pouco explorado e que requer investigação com mais detalhes para definição de como será realizada sua implementação.

Ainda existem questões sem definições claras mas que já foram mapeadas para implementação, incluem: pontos de melhoria não críticos ao código, infraestrutura da hospedagem da API em *cloud*, automatização da compilação em cada mudança no repositório e acionamento da execução dos testes (unitários e de integração) após cada compilação.

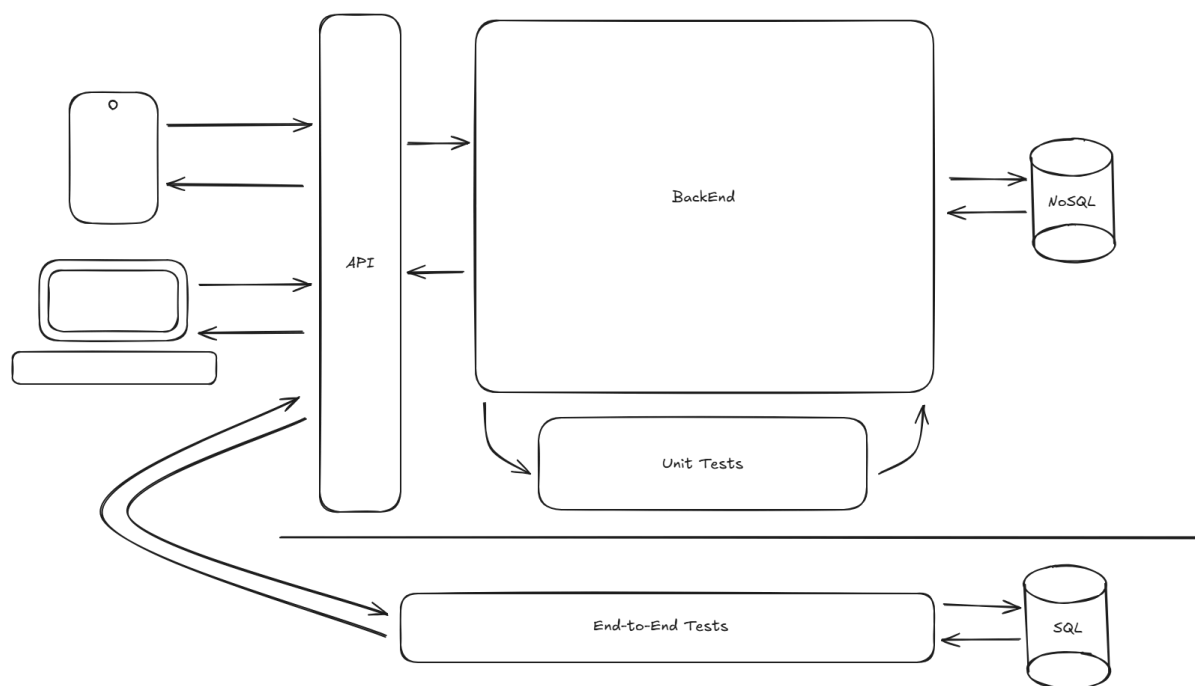
Resultados Preliminares

O presente trabalho foi estruturado no formato de uma API RESTful, que possibilita uma interação agnóstica quanto à origem da requisição. A restrição estabelecida para utilização do software é que todo código fonte a ser analisado esteja contido e publicado em um repositório no *GitHub*. Um exemplo é o próprio repositório deste projeto, que pode ser acessado em: <https://github.com/HugorTorquato/MBA-TCC>. Uma vez contido no repositório, o link da estrutura de pasta que deseja ser feita a análise é utilizado como argumento de entrada para os *endpoints* da API. Após o envio da requisição, a lógica implementada vai acessar o repositório, baixar os arquivos que contém a implementação à ser avaliada e processar o conteúdo. Como resultado, um arquivo JSON estruturado com a relação entre definição de classes é enviado para a origem da requisição do *endpoint*.

O diagrama apresentado na Figura 2 apresenta a completude do projeto, desde o processo de requisição de análise ao *endpoint* até o armazenamento das informações em bancos de dados. O fluxo do diagrama pode ser dividido em duas partes: fluxo funcional e fluxo de validação. O primeiro representa o ponto de vista de um usuário, em que será feita uma requisição para a API por meio de um dispositivo externo, seja ele um celular ou computador. A requisição é recebida e já aciona toda a lógica do *backend* associada, seja para coletar os dados de uma nova estrutura de pastas ou acessar determinadas

informações já carregadas e contidas no banco de dados. Ou seja, a lógica do *backend* que vai ser executada depende do que está sendo solicitado da API.

De forma procedural, o *backend* é responsável tanto por receber uma nova estrutura de pastas, baixar os arquivos que contém o código fonte para análise, extrair as informações de maneira estruturada, armazená-las em um banco de dados e possibilitar a coleta dessas informações de acordo com o que deseja ser coletado pela API. Por fim, a informação é estruturada em um formato JSON e retornada ao usuário. Toda essa explicação pode ser observada na Figura 2 como uma requisição chegando ao bloco da API, sendo processada e acionando a lógica no bloco *backend* que, por sua vez, vai salvar ou consultar dados no banco de dados NoSQL. E o processo inverso é estabelecido para retornar o resultado ao requerente, banco de dados para o *backend* que envia as informações para a camada de API que retorna à origem da requisição.



Visão de alto nível do funcionamento do software proposto

Figura 2. Esquemático da visão de alto nível do funcionamento do software proposto.

Fonte: Elaborado pelo autor.

O segundo fluxo, de validação, é exercitado somente durante a etapa de implementação do software. À medida que a lógica se expande e novos requisitos são implementados, é de extrema importância garantir seu funcionamento, além de garantir que nenhuma nova implementação altera comportamentos já esperados e validados. Nesse contexto, os testes apresentam um papel crucial no projeto.

Foram implementadas duas camadas de testes com o objetivo de cobrir o máximo possível da lógica e garantir sua confiabilidade. A primeira camada consiste nos testes unitários, que garantem o funcionamento dos métodos contidos em cada classe do projeto de forma, quase, isolada. Estes testes dependem somente daquela porção do código para serem exercidas e as respectivas dependências. Pelo fato de serem alto contidos, são testes considerados rápidos e de execução com baixo custo computacional, ideal para abranger a completude da implementação, embora não considerem as interações entre os diferentes módulos. Até o presente estado de desenvolvimento do projeto, 78 testes unitários foram implementados e são executados em uma média de 1.5 segundos.

A segunda camada já compreende os chamados testes funcionais, *end-to-end*, responsáveis por garantir a funcionalidade da aplicação como um todo, simulando um uso real. Nesse caso, uma entrada conhecida é fornecida à API, e o retorno é avaliado com base no resultado esperado. No diagrama de Figura 2, a interação entre estes testes com o *software* principal segue o mesmo fluxo que um usuário teria: é enviado uma requisição para a API que vai acionar a respectiva lógica no backend e prover uma resposta. Estes são considerados testes lentos e custosos, ao passo que exercitam todo o fluxo da aplicação por depender da resposta final. Até o presente estado de desenvolvimento do projeto, 5 testes de integração foram implementados e são executados em uma média de 11 segundos. A diferença de tempo em relação aos testes unitários é significativa, e por isso, há um maior número de testes unitários no projeto.

Em detaque o bloco da API, os *endpoints* disponíveis até o presente momento são: *listFilesInUrl*, *downloadFilesInUrl* e *downloadAndRetrieveSourceFileContent*. O *listFilesInUrl* retorna, no formato JSON, uma lista de arquivos e pastas contidos no repositório *GitHub* informado, com um filtro somente para arquivos no formato .cpp e .h.

O *endpoint* *downloadFilesInUrl* é responsável por baixar os arquivos contidos no repositório *GitHub* em uma pasta temporária local no servidor. Estes precisam ser processados e analisados antes de serem descartados. Embora seja possível manter todo o conteúdo em memória, essa abordagem foi descartada devido às limitações físicas do servidor. Trata-se de uma decisão arquitetural que ainda precisa ser metrificada. Esse *endpoint* retorna um dicionário, em que a chave é o nome do arquivo com seu caminho relativo e o valor é o próprio caminho relativo do arquivo.

O último *endpoint* implementado é o *downloadAndRetrieveSourceFileContent*, também responsável por baixar os arquivos contidos no repositório *GitHub*, mas este retorna o conteúdo de cada arquivo processado. O retorno é estruturado no formato chave valor, como chave o nome do arquivo seguido do caminho relativo e como valor o conteúdo no formato de sequência de caracteres. O resultado é ainda um dado bruto de código, mas este

é a entrada para o início da etapa de *scanner* do código fonte para identificar e agrupar os *tokens*.

O processo de desenvolvimento segue uma abordagem iterativa baseada na implementação e validação de cada *endpoint* da API, etapa por etapa, garantindo seu funcionamento por meio de testes. Muitos dos componentes descritos serão de utilização interna para validação dos componentes, principalmente durante a execução dos testes de integração. Entretanto, os *endpoints* exercitam a mesma porção do código que é utilizada no fluxo funcional da aplicação, mesmo que em etapas menores.

Esse ponto levanta uma questão importante: a implementação está orientada à funcionalidade ou aos testes? O foco em garantir o correto funcionamento do sistema por meio da implementação de testes exerceu grande influência nas decisões arquiteturais do projeto. Um exemplo é o acesso externo a partes isoladas da lógica, presente na criação dos *endpoints*, e a ampla utilização do padrão de projeto *strategy*, que facilita a injeção de dependências e, conseqüentemente, a criação de *mocks* nos testes unitários. Entretanto, esse exemplo traz benefícios não apenas para os testes, mas também se alinha a outros princípios adotados, como o princípio da responsabilidade única de uma classe ou método e à reutilização de código sempre que possível. Todas essas são práticas que fazem parte das boas práticas na engenharia de *software*, mas que exercem influência direta na facilidade, ou dificuldade, de implementar testes e na escalabilidade do projeto, seja para expansão ou usabilidade. Um ponto de atenção, no entanto, é que a arquitetura do *software* deve ser eficiente e centrada na solução proposta, não pode ser alterada unicamente em função da implementação de testes. Durante a implementação do projeto, percebeu-se o quão fácil é ultrapassar esse limite. Como resultado, manter um foco conjunto em aplicação e testes permite extrair benefícios de ambas as partes: escalabilidade e confiabilidade de implementação.

Referências

Aho, A.V.; Lam, M.S.; Sethi, R.; Ullman, J.D. 2007. *Compilers: Principles, Techniques, & Tools*. Segunda Edição. Pearson Education, Inc, Boston, Massachusetts, Estados Unidos.

Ceconello, J. 2016. *Ferramenta de auxílio à análise de anomalias em códigos Orientados a Objeto*. Monografia em Ciência da Computação. Universidade Federal do Rio Grande do Sul, Porto Alegre, Rio Grande do Sul, Brasil.

Cooper, K.D; Torczon,L. 2012. Engineering a Compiler. Segunda Edição. ELSEVIER, Huston, Texas, Estados Unidos.

Dean,J; Grove,D; Chambers,C. 1995. Optimization of Object-Oriented Programs Using Static Class Hierarchy Analysis. 9th European Conference: 77-101.

Iglberger, K. 2022. C++ Software Design: Design Principles and Patterns for High-Quality Software. O'Reilly Media, Inc, Sebastopol, Califórnia, Estados Unidos.

Langr, J. 2013. Modern C++ Programming with Test-Driven Development. The Pragmatic Programmers, LLC. Colorado Springs, Colorado, Estados Unidos.

Martin, R.C. 2009. Clean Code A Handbook of Agile Software Craftsmanship. Pearson Education, Inc, Boston, Massachusetts, Estados Unidos.

Nystrom, R. 2015-2020. Crafting Interpreters. Genever Benning. Seattle, Washington, Estados Unidos.

Silva, D; Samarasekara, H.M.P.P.K.H; Hettiarachchi, R.T; Senanayake, W.A.B.M; Sanjaya, A.M.T; Abeysekara, M.P. 2023. A Comparative Analysis of Static and Dynamic Code Analysis Techniques. TechRxiv: DOI: 10.36227/techrxiv.22810664.v1.

Wang, S; Ding, L; Shen, L; Luo, Y; Du, B; Tao, D. 2024. Título do trabalho. Findings of the Association for Computational Linguistics: 13619–13639.